DSA PATTERN REFERENCE

Sliding Window

Recognition cues, invariants, Python templates, and an 8-problem practice ladder

Use sliding window when the answer is a contiguous range and one boundary move can update validity without rescanning the range.

VERSION 0.1-pilot

CODE LANGUAGE PYTHON 3

STATUS HUMAN REVIEW REQUIRED

DSA / SLIDING WINDOW

01 RECOGNIZE THE PATTERN

At a glance

CORE SIGNAL

Use sliding window when the answer is a contiguous range and one boundary move can update validity without rescanning the range.

Strong cues

+ The prompt asks about a contiguous subarray or substring.
+ The objective is longest, shortest, count, maximum, or minimum over a range.
+ One incoming and one outgoing value can update the state incrementally.
+ A brute-force solution repeats nearly identical work across overlapping ranges.

When not to force it

+ The chosen elements can be non-contiguous, which usually points to subsequence logic or dynamic programming.
+ Moving a boundary does not change validity monotonically. Negative numbers often break sum-based shrinking.
+ The required state cannot be updated more cheaply than rescanning the entire range.

Choose the variant

Question	If yes	If no
Is the window length exactly k?	Use the fixed-size template.	Continue to a validity-driven window.
Do you need the longest valid range?	Shrink while invalid; record after validity is restored.	For the shortest satisfying range, record while valid and then shrink.

Working invariant

For an inclusive window, the state must describe exactly the values from left through right. Every add and remove operation must preserve that statement.

02.1 REUSABLE TEMPLATE

Fixed-size window

INVARIANT

After eviction, the current window contains at most k items; an answer is recorded only when it contains exactly k.

```
def max_sum_of_k(nums, k):
    if k <= 0 or k > len(nums):
        raise ValueError("k must be in [1, len(nums)]")

    left = 0
    window_sum = 0
    best = float("-inf")

    for right, value in enumerate(nums):
        window_sum += value

        if right - left + 1 > k:
            window_sum -= nums[left]
            left += 1

        if right - left + 1 == k:
            best = max(best, window_sum)

    return best
```

Use it for

Maximum or minimum sum/average over exactly k values; frequency matching for an exact-length substring; deque-based extrema over each range of size k.

02.2 REUSABLE TEMPLATE

Longest valid variable window

INVARIANT

After the while-loop, the window satisfies the validity rule.

```
def longest_at_most_k_distinct(values, k):
    if k < 0:
        raise ValueError("k cannot be negative")

    left = 0
    counts = {}
    best = 0

    for right, value in enumerate(values):
        counts[value] = counts.get(value, 0) + 1

        while len(counts) > k:
            outgoing = values[left]
            counts[outgoing] -= 1
            if counts[outgoing] == 0:
                del counts[outgoing]
            left += 1

        best = max(best, right - left + 1)

    return best
```

Answer placement

Shrink while the window is invalid. Record only after the loop, when the invariant is true again.

02.3 REUSABLE TEMPLATE

Shortest satisfying variable window

INVARIANT
Every candidate is recorded while the sum meets the target; positive inputs make leftward shrinking monotonic.

```
def min_len_at_least(target, nums):
    left = 0
    window_sum = 0
    best = float("inf")

    for right, value in enumerate(nums):
        window_sum += value

        while window_sum >= target:
            best = min(best, right - left + 1)
            window_sum -= nums[left]
            left += 1

    return 0 if best == float("inf") else best
```

Answer placement

Record inside the valid shrink loop before removing the left value. The outgoing value may make the next window invalid.

ASSUMPTION
This exact positive-sum version is not generally valid when negative numbers are allowed.

03 TRACE THE STATE

Find the shortest subarray with sum at least 7.

EXAMPLE

nums = [2, 3, 1, 2, 4, 3] | Answer: 2, from [4, 3]

R	Added	Window before shrink	Recorded	Window after shrink
0	2	[2] / sum 2	-	[2]
1	3	[2,3] / sum 5	-	[2,3]
2	1	[2,3,1] / sum 6	-	[2,3,1]
3	2	[2,3,1,2] / sum 8	4	[3,1,2] / sum 6
4	4	[3,1,2,4] / sum 10	4, then 3	[2,4] / sum 6
5	3	[2,4,3] / sum 9	3, then 2	[3] / sum 3

What the trace proves

Every recorded range meets the target. Repeated shrinking tests all shorter candidates ending at the same right index. Because the inputs are positive, once a removal breaks the target, more removals cannot restore it.

04 DEBUG AND EXPLAIN

Failure-mode checklist

Symptom	Check first
The answer is off by one.	Use right - left + 1 for an inclusive window and trace the first full window.
A supposedly valid answer violates the rule.	Record only after the invalid-shrink loop for longest-valid problems.
The state drifts away from the actual range.	Undo every outgoing value before incrementing left.
The algorithm misses a smaller valid range.	Use while, not if, when repeated shrinking can improve or restore the window.
A sum-based solution fails with negative numbers.	The monotonic assumption is false; consider prefix sums with a deque or another method.
The code times out despite two pointers.	Avoid rescanning the window or its whole frequency map after each move.

Complexity proof

Time

+ Usually O(n): right advances at most n times and left advances at most n times.

Space

+ O(1) for a running scalar; O(u) for a map with u distinct values; O(k) for a fixed-window deque.

INTERVIEW PROOF

Nested syntax does not imply O(n squared). Count total pointer moves across the whole run.

05 PRACTICE LADDER

Eight problems, one progression

Solve in order. Before coding, label the variant, state, invariant, shrink rule, and answer position.

#	Problem	Level	Primary focus
1	**Maximum Average Subarray I**	Easy	Fixed-size sum
2	**Minimum Size Subarray Sum**	Medium	Shortest satisfying window
3	**Longest Substring Without Repeating Characters**	Medium	Frequency counts and longest valid window
4	**Max Consecutive Ones III**	Medium	Budgeted violations
5	**Fruit Into Baskets**	Medium	At most k distinct values
6	**Permutation in String**	Medium	Fixed-size frequency matching
7	**Minimum Window Substring**	Hard	Required frequencies and shortest valid window
8	**Sliding Window Maximum**	Hard	Monotonic deque state

Re-solve rule

A problem counts as learned only when you can re-solve it from a blank editor after at least 48 hours and explain the invariant before implementation.

06 INTERVIEW-READY RECAP

Five-sentence talk track

- [ ] The objective is over a contiguous range, and brute force repeats work across overlapping ranges.
- [ ] I will track [state] for the inclusive window from left to right.
- [ ] After the shrink loop, my invariant is [validity statement].
- [ ] I update the answer [before/during/after] shrinking because [eligibility reason].
- [ ] Each pointer only moves forward, so total pointer work is O(n).

Source and review note

Problem titles and links were verified against the official LeetCode problem pages on 2026-07-17. Explanations and templates are original InterviewRunway draft content.

PUBLISHING GATE
Human technical review is required before publishing. Compile and test all code during the pilot QA pass. Confirm the launch language strategy before generating the full DSA set.

Fast recall

REMEMBER
CONTIGUOUS / INCREMENTAL STATE / EXPLICIT INVARIANT / CORRECT ANSWER MOMENT